Claude Certified Architect

Foundations (CCAR-F) — Focused Study Guide

Distilled to the decision rules, anti-patterns, and scenarios the exam actually tests

60 items · 4 of 6 scenarios · 120 min · Pass = 720/1000

Domains: Agentic Architecture (27%) · Tool & MCP (18%) · Claude Code (20%)

Prompt & Structured Output (20%) · Context & Reliability (15%)

How to Read This Exam

The exam almost never asks "what is X." It asks "given a broken production system, what is the *most effective* / *first* / *most maintainable* fix." Most wrong answers are technically plausible but violate one of the meta-rules below. Learn these first — they decide most of the marginal questions.

> **Meta-rule 1 — Deterministic requirement → programmatic enforcement, not prompts.** If the requirement is "this must always happen" (identity verification before a refund, block refunds over $500, force a tool to run first), the answer is a **hook / prerequisite gate / forced tool_choice**. Prompt instructions have a non-zero failure rate and are always wrong when money, safety, or compliance is at stake.

> **Meta-rule 2 — Proportionate first response.** When asked for the "first step" or "most effective" fix, prefer the **lowest-effort change that addresses the root cause**: fix tool descriptions, add explicit criteria, add 2–4 few-shot examples. Reject over-engineered answers (train a classifier, build an ML routing layer, deploy separate models) when a prompt/description fix hasn't been tried.

> **Meta-rule 3 — Tool descriptions are the primary tool-selection mechanism.** Misrouting between similar tools is almost always fixed by **rewriting descriptions** (inputs, example queries, edge cases, when-to-use-vs-alternative) — before few-shot, before routing layers, before consolidation.

> **Meta-rule 4 — Self-reported confidence and sentiment are unreliable proxies.** An LLM's own confidence score is poorly calibrated (it is confidently wrong on hard cases). Customer sentiment does not correlate with case complexity. Any answer that routes/escalates based on these is wrong.

> **Meta-rule 5 — Least privilege for tools.** Too many tools (18 vs 4–5) degrades selection. Agents misuse tools outside their role. Give each agent only its tools; add a **narrow scoped tool** for a high-frequency cross-role need rather than all tools.

> **Meta-rule 6 — Bigger context window does not fix attention quality.** Inconsistent/contradictory review across many files is **attention dilution**, solved by splitting into per-file passes + an integration pass — not a larger model or larger context.

> **Meta-rule 7 — Structured data beats prose everywhere.** Structured error context (not "operation failed"), structured claim–source mappings (not summarized prose), structured case-facts blocks (not vague summaries), structured handoff summaries. Whenever an answer preserves structure/metadata over free text, it is usually right.

> **Meta-rule 8 — Diagnose the actual failing component.** If logs show the coordinator decomposed the topic badly, the fault is the coordinator, not the downstream subagents that executed their (wrong) assignments correctly. Match the fix to the component the evidence implicates.

Non-existent features (auto-eliminate these distractors)

Fake option seen in questions	Reality
`CLAUDE_HEADLESS=true` env var	Not real. Non-interactive mode = `-p` / `--print`.
`--batch` flag on Claude Code CLI	Not real. Batching is the Message Batches API, not a CLI flag.
`.claude/config.json` with a commands array	Not real. Slash commands live as files in `.claude/commands/`.
Redirect `< /dev/null` as the CI fix	A Unix workaround, not the documented answer; use `-p`.

Domain 1 · Agentic Architecture & Orchestration (27%)

Largest domain. Master the agentic loop, coordinator–subagent orchestration, enforcement/handoff, hooks, decomposition, and session management.

1.1 The Agentic Loop

- Loop control is driven **solely by** `stop_reason`: `"tool_use"` → execute the requested tool(s), append results, iterate. `"end_turn"` → terminate and present the final answer.
- Tool results are **appended to conversation history** so the model reasons about the next step with the new information.
- Model-driven decisions (Claude chooses the next tool from context) ≠ pre-configured decision trees / fixed tool sequences.

> **Anti-patterns (always wrong):** parsing natural-language text to decide the loop is done; using an arbitrary iteration cap as the primary stop mechanism; checking for assistant text content as a "completion" signal. Termination = `stop_reason == "end_turn"`, nothing else.

1.2 Coordinator–Subagent Orchestration (hub-and-spoke)

- **Coordinator owns all inter-agent communication**, error handling, routing — for observability and consistent handling.
- Subagents have **isolated context**; they do NOT inherit the coordinator's history automatically.
- Coordinator responsibilities: task decomposition, delegation, result aggregation, and **dynamically choosing which subagents to invoke** by query complexity (not always running the full pipeline).
- Partition scope across subagents (distinct subtopics / source types) to minimize duplication.
- Run **iterative refinement**: coordinator evaluates synthesis for gaps, re-delegates targeted queries, re-synthesizes until coverage is sufficient.

> **Key failure: overly narrow task decomposition** → incomplete coverage of a broad topic (e.g. "creative industries" decomposed into only visual-arts subtopics). The fault is the coordinator's decomposition, not the subagents.

1.3 Subagent Invocation & Context Passing

- Subagents are spawned via the `Task` **tool**; the coordinator's `allowedTools` **must include** `"Task"`.
- Context must be **explicitly placed in the subagent's prompt** — no automatic inheritance, no shared memory across invocations. Pass prior findings (search results, doc analysis) directly into the synthesis subagent's prompt.
- Use **structured formats** to separate content from metadata (source URLs, doc names, page numbers) to preserve attribution.
- **Parallel spawning:** emit *multiple* `Task` *calls in a single response* (not across separate turns).
- `AgentDefinition` sets each subagent's description, system prompt, and tool restrictions.
- Write coordinator prompts as **goals + quality criteria**, not step-by-step procedures, so subagents can adapt.

1.4 Multi-step Workflows: Enforcement & Handoff

- **Programmatic enforcement** (hooks, prerequisite gates) vs prompt-based guidance. When deterministic compliance is required, prompts alone have non-zero failure → use gates.
- Example: block `process_refund` until `get_customer` has returned a verified customer ID.
- Decompose multi-concern requests into distinct items, investigate in parallel on shared context, then synthesize one unified resolution.
- **Structured handoff** on escalation: customer ID, root cause, refund amount, recommended action — because the human lacks the conversation transcript.

1.5 Agent SDK Hooks

Hook use	Purpose
`PostToolUse`	Intercept tool *results* to transform/normalize before the model sees them (e.g. unify Unix timestamps, ISO-8601, numeric status codes from different MCP tools).
Tool-call interception	Intercept outgoing tool *calls* to enforce policy (block refunds > $500, redirect to human escalation).

> Choose **hooks over prompts** whenever business rules require a guaranteed outcome. Hooks = deterministic; prompts = probabilistic.

1.6 Task Decomposition Strategy

Pattern	Use when	Example
Fixed sequential (prompt chaining)	Predictable, multi-aspect, known steps	Analyze each file individually, then a cross-file integration pass
Dynamic / adaptive decomposition	Open-ended investigation; subtasks depend on findings	"Add comprehensive tests to a legacy codebase": map structure → find high-impact areas → adaptive prioritized plan

1.7 Session State, Resumption, Forking

- `--resume <session-name>` continues a specific named prior conversation.
- `fork_session` branches from a shared analysis baseline to explore divergent approaches (e.g. compare two refactoring strategies).
- When resuming after code changed, **tell the agent which files changed** for targeted re-analysis.
- **Resume vs fresh start:** resume when prior context is mostly valid; start fresh with an injected structured summary when prior tool results are *stale* (more reliable than resuming stale results).

Domain 2 · Tool Design & MCP Integration (18%)

2.1 Effective Tool Interfaces

- Tool descriptions are the **primary selection signal**. Minimal descriptions ("Retrieves customer information" / "Retrieves order details") cause misrouting between similar tools.
- Good descriptions include: **input formats, example queries, edge cases, boundaries, and when to use this vs a similar tool.**
- Fixes for overlap: **rename + re-describe** (`analyze_content` → `extract_web_results`); **split generic tools** into purpose-specific ones (`analyze_document` → `extract_data_points`, `summarize_content`, `verify_claim_against_source`).
- Review system prompts for **keyword-sensitive wording** that can override good descriptions and create unintended tool associations.

2.2 Structured Error Responses (MCP)

- Use the MCP `isError` **flag** to signal failures back to the agent.
- Return structured metadata: `errorCategory` (transient / validation / business / permission), `isRetryable` boolean, human-readable description.
- Business-rule violations: include `retriable: false` + a customer-friendly explanation so the agent communicates appropriately.
- Distinguish **access failures** (need retry decision) from **valid empty results** (successful query, no matches).

Error type	Retryable?	Agent action
Transient (timeout, service down)	Yes	Retry / try alternative
Validation (bad input)	Sometimes (fix input)	Correct and resend
Business (policy violation)	No	Explain to user / escalate
Permission	No	Escalate / request access

> **Anti-pattern:** uniform "Operation failed" responses — they strip the metadata the agent needs to recover and cause wasted retries on non-retryable errors.

2.3 Tool Distribution & `tool_choice`

- Too many tools degrades selection. Scope each agent to its role; add limited cross-role tools only for high-frequency needs.
- Replace generic tools with constrained ones (`fetch_url` → `load_document` that validates document URLs).
- Give the synthesis agent a narrow `verify_fact` tool for the common case; route complex cases through the coordinator.

`tool_choice`	Behavior	Use when
`"auto"`	Model may call a tool or return text	Normal operation
`"any"`	Must call some tool, model picks which	Guarantee structured output; prevent conversational text; unknown document type among multiple schemas
Forced `{"type":"tool","name":"..."}`	Must call that specific tool	Ensure a tool runs first (e.g. `extract_metadata` before enrichment), then continue in follow-up turns

2.4 Integrating MCP Servers

Scope	File	Meaning
Project / shared team	`.mcp.json`	Version-controlled, shared with the whole team
User / personal / experimental	`~/.claude.json`	Personal, not shared

- Use **env-var expansion** in `.mcp.json` (`${GITHUB_TOKEN}`) to avoid committing secrets.
- Tools from **all configured servers are discovered at connection time** and available simultaneously.
- **MCP resources** expose content catalogs (issue summaries, doc hierarchies, DB schemas) to reduce exploratory tool calls. Rule of thumb: *resources = read-only content/catalogs; tools = actions.*
- Enrich MCP tool descriptions so the agent prefers capable MCP tools over built-ins (e.g. over `Grep`).
- Prefer existing community MCP servers for standard integrations (e.g. Jira); build custom only for team-specific workflows.

2.5 Built-in Tools (Read, Write, Edit, Bash, Grep, Glob)

Tool	Use for
`Grep`	Search file *contents* for patterns (function callers, error strings, imports)
`Glob`	Find files by *name/extension* pattern (`**/*.test.tsx`)
`Read` / `Write`	Full-file operations
`Edit`	Targeted edits via *unique* text match

> **Edit fallback:** when `Edit` fails on a non-unique match, use `Read` + `Write` . **Explore incrementally:** `Grep` entry points → `Read` to follow imports/trace flows — do not read every file upfront.

Domain 3 · Claude Code Configuration & Workflows (20%)

3.1 CLAUDE.md Hierarchy

Level	Location	Shared via VCS?
User	`~/.claude/CLAUDE.md`	No — personal only
Project	`.claude/CLAUDE.md` or root `CLAUDE.md`	Yes — whole team
Directory	subdirectory `CLAUDE.md`	Yes, but only for that directory tree

- Classic bug: a teammate isn't getting instructions because they live in **user-level** config → move to **project-level**.
- `@import` keeps CLAUDE.md modular (pull in relevant standards files per package).
- `.claude/rules/` holds topic-specific rule files (testing.md, api-conventions.md) as an alternative to a monolithic file.
- `/memory` command verifies which memory files are loaded (diagnose inconsistent behavior).

3.2 Slash Commands & Skills

Artifact	Project (shared)	User (personal)
Slash commands	`.claude/commands/`	`~/.claude/commands/`
Skills	`.claude/skills/` (SKILL.md)	`~/.claude/skills/` (use a different name to avoid affecting teammates)

SKILL.md frontmatter options:

- `context: fork` — run the skill in an **isolated sub-agent context** so verbose/exploratory output doesn't pollute the main conversation.
- `allowed-tools` — restrict tools during skill execution (e.g. only file-write to prevent destructive actions).
- `argument-hint` — prompt the developer for required params when invoked without arguments.

> **Skills vs CLAUDE.md:** skills = on-demand, task-specific workflows (invoked). CLAUDE.md = always-loaded universal standards.

3.3 Path-Specific Rules

- `.claude/rules/` files use YAML frontmatter `paths:` with glob patterns (`paths: ["terraform/**/*"]` , `["**/*.test.tsx"]`).
- Rules load **only when editing matching files** → less irrelevant context, fewer tokens.
- **Choose glob rules over subdirectory CLAUDE.md** when a convention applies to a file *type spread across many directories* (test files next to their components). Subdirectory CLAUDE.md is directory-bound and can't handle scattered files.

3.4 Plan Mode vs Direct Execution

Plan mode	Direct execution
Large-scale changes, many files	Simple, well-scoped, one function
Multiple valid approaches	Clear single approach
Architectural decisions (service boundaries, library migration affecting 45+ files)	Single-file bug fix with a clear stack trace; add a validation conditional
Explore/design before committing → prevent costly rework	Scope already understood

- Combine: plan mode to *investigate* a migration, direct execution to *implement* the planned approach.
- **Explore subagent**: isolate verbose discovery, return a summary → protect main context.

> Wrong answers: "start direct, switch to plan only if complexity emerges" — the complexity is *already stated in the requirements*. Don't defer.

3.5 Iterative Refinement

- **Concrete input/output examples** (2–3) are the best way to communicate a transformation when prose is interpreted inconsistently.
- **Test-driven iteration:** write the test suite first, then iterate by sharing test failures.
- **Interview pattern:** have Claude ask questions to surface considerations (cache invalidation, failure modes) before implementing in an unfamiliar domain.
- **Single message vs sequential:** fix *interacting* problems together in one detailed message; fix *independent* problems sequentially.

3.6 Claude Code in CI/CD

- `-p` / `--print` = non-interactive mode (prevents hangs waiting for input). This is *the* CI answer.
- `--output-format json` + `--json-schema` = machine-parseable structured findings for inline PR comments.
- CLAUDE.md supplies project context to CI-invoked Claude (testing standards, fixtures, review criteria).
- Include prior review findings when re-running after new commits; instruct Claude to report only new/unaddressed issues (avoid duplicate comments).
- Provide existing test files so generation avoids duplicate scenarios.
- **Session isolation:** the session that *generated* code is worse at reviewing it → use an independent review instance.

Domain 4 · Prompt Engineering & Structured Output (20%)

4.1 Explicit Criteria to Cut False Positives

- Explicit categorical criteria beat vague instructions. **"Flag a comment only when its claimed behavior contradicts the actual code behavior"** > **"check that comments are accurate."**
- "Be conservative" / "only high-confidence findings" do **not** improve precision — use specific categories instead.
- High false-positive categories erode trust in the accurate ones. You may **temporarily disable a noisy category** while you improve its prompt.
- Define severity with **concrete code examples per level** for consistent classification.

4.2 Few-Shot Prompting

- Best technique for **consistent format + judgment on ambiguous cases** when instructions alone are inconsistent.
- Use 2–4 targeted examples that **show the reasoning** for why one action beat a plausible alternative.
- Few-shot enables **generalization to novel patterns** (not just matching the listed cases) and reduces extraction hallucination on varied document structures.

> **Descriptions vs few-shot:** for tool *misrouting*, fix descriptions *first* (root cause, low token cost). Few-shot is the fix for *ambiguous judgment/format consistency*, not for compensating for thin descriptions.

4.3 Structured Output via Tool Use + JSON Schema

- `tool_use` **with a JSON schema is the most reliable way to guarantee schema-compliant output** — eliminates JSON *syntax* errors.
- It does **not** prevent *semantic* errors (line items not summing to total, values in the wrong field).
- Make fields **optional / nullable** when the source may lack them → prevents the model fabricating values to satisfy required fields.
- Use `enum` with an `"other"` + detail string for extensible categories; add `"unclear"` for ambiguous cases.
- Put format-normalization rules in the prompt alongside the strict schema to handle messy source formats.
- `tool_choice`: `"any"` when multiple schemas exist and the doc type is unknown; forced when a specific extraction must run first.

4.4 Validation, Retry, Feedback Loops

- **Retry-with-error-feedback:** resend the original document + failed extraction + the specific validation error for self-correction.
- Retries fix **format/structural** errors. Retries do **NOT** help when the information is simply *absent from the source* (e.g. it lives in an external document not provided).
- Self-correction design: extract `calculated_total` alongside `stated_total` to flag mismatches; add `conflict_detected` booleans for inconsistent source data.
- `detected_pattern` field on findings enables analysis of which constructs trigger dismissed false positives.

4.5 Batch Processing (Message Batches API)

Property	Detail
Cost	50% savings
Latency	Up to 24h window, **no guaranteed SLA**
Correlation	`custom_id` pairs requests to responses
Limitation	**No multi-turn tool calling** within a single request

- **Good fit:** non-blocking, latency-tolerant — overnight reports, weekly audits, nightly test generation.
- **Bad fit:** blocking workflows — pre-merge checks where a developer waits.
- Handle failures by resubmitting only failed `custom_id`s (e.g. chunk docs that exceeded context).
- Refine the prompt on a *sample* before batching large volumes; compute submission cadence from the SLA (e.g. 4-hour windows to guarantee a 30-hour SLA with 24h processing).

> Wrong: switch *both* blocking and overnight workflows to batch; "batch is often faster so it's fine for blocking"; avoid batch because of "result ordering" (false — `custom_id` handles correlation).

4.6 Multi-Instance / Multi-Pass Review

- A model retains its generation reasoning, so it is **worse at questioning its own output** in the same session. An **independent review instance** beats self-review instructions or extended thinking.
- **Multi-pass review:** per-file passes for local issues + a separate cross-file integration pass. Fixes attention dilution and contradictory findings.
- Optionally have the model self-report confidence per finding to route review attention.

> Wrong for a diluted 14-file review: bigger context window (doesn't fix attention); force developers to split PRs (shifts burden); 2-of-3 consensus voting (suppresses real bugs caught only intermittently).

Domain 5 · Context Management & Reliability (15%)

5.1 Preserving Critical Info Across Long Interactions

- **Progressive summarization risk:** numbers, %, dates, and customer-stated expectations get flattened into vague summaries. Extract them into a persistent **"case facts" block** included in every prompt, *outside* the summarized history.
- **Lost-in-the-middle:** models handle the start and end of long inputs reliably, may drop the middle → put key findings **at the beginning**, use explicit section headers.
- **Trim verbose tool outputs** to only relevant fields *before* they accumulate (keep ~5 relevant fields, not 40+ per order lookup).
- Pass complete conversation history in each request for coherence; but downstream agents with small budgets should receive **structured facts/citations**, not verbose reasoning chains.

5.2 Escalation & Ambiguity Resolution

- Valid escalation triggers: **customer asks for a human**; **policy exception/gap** (not merely "complex"); **inability to make meaningful progress**.
- Honor an explicit human request **immediately** — don't investigate first. If the customer is frustrated but the issue is within capability, acknowledge and offer resolution; escalate only if they reiterate.
- Escalate when policy is **silent/ambiguous** on the request (e.g. competitor price-matching when policy only covers own-site adjustments).
- Multiple record matches → **ask for another identifier**, never pick by heuristic.
- Fix miscalibrated escalation with **explicit criteria + few-shot examples** first (proportionate). Not self-confidence scores, not sentiment, not a trained classifier.

5.3 Error Propagation in Multi-Agent Systems

- Return **structured error context**: failure type, attempted query, partial results, alternative approaches → enables intelligent coordinator recovery.
- Distinguish access failures (need retry decision) from valid empty results (no matches).
- Subagents do **local recovery** for transient failures; propagate only what they can't resolve, with what was attempted + partial results.
- Annotate synthesis with coverage gaps (well-supported vs missing due to unavailable sources).

> **Both anti-patterns:** silently suppressing errors (empty result marked success) AND terminating the whole workflow on a single failure. Also wrong: generic "search unavailable" that hides context from the coordinator.

5.4 Context in Large Codebase Exploration

- Extended sessions degrade: the model gives inconsistent answers and cites "typical patterns" instead of specific classes it found earlier.
- **Scratchpad files** persist key findings across context boundaries; reference them for later questions.
- **Subagent delegation** isolates verbose exploration; main agent keeps high-level coordination.
- Summarize a phase before spawning subagents for the next; inject summaries into initial context.
- Crash recovery: each agent exports state to a known location; coordinator loads a **manifest** on resume and injects it into prompts.
- `/compact` reduces context usage during long exploration.

5.5 Human Review & Confidence Calibration

- Aggregate accuracy (e.g. 97%) can **mask** poor performance on specific document types or fields. Validate accuracy **by document type and field** before reducing human review.
- **Stratified random sampling** of high-confidence extractions measures ongoing error rates and catches novel patterns.
- Calibrate **field-level confidence** using a labeled validation set; route low-confidence or ambiguous/contradictory sources to humans.

5.6 Provenance & Uncertainty in Multi-Source Synthesis

- Attribution is lost during summarization when claim–source mappings aren't preserved. Require subagents to output **structured claim–source mappings** (URLs, doc names, excerpts) that downstream agents preserve through synthesis.
- **Conflicting stats from credible sources:** annotate the conflict *with source attribution*; don't arbitrarily pick one. Complete analysis with both values included and flagged; let the coordinator reconcile.
- Require **publication/collection dates** so temporal differences aren't misread as contradictions.
- Structure reports to separate **well-established from contested** findings; render content by type (financial → tables, news → prose, technical → structured lists), not one uniform format.

Sample Questions — Answer Logic

The reasoning matters more than the answer letter. For each, note the meta-rule and why the distractors fail.

Q1 · SUPPORT Agent skips `get_customer`, refunds wrong accounts. Fix?
A — programmatic prerequisite blocking `lookup_order` / `process_refund` until `get_customer` returns a verified ID.
Meta-rule 1: financial consequence → deterministic gate. B/C rely on probabilistic prompt/few-shot compliance. D fixes tool *availability*, not tool *ordering*.

Q2 · SUPPORT Misrouting between `get_customer` and `lookup_order` (both minimal descriptions). First step?
B — expand each tool's description (inputs, example queries, edge cases, when-to-use-vs-similar).
Meta-rule 3: descriptions are the root cause. A adds tokens without fixing it. C over-engineers. D (consolidate) is more effort than a "first step."

Q3 · SUPPORT 55% resolution; escalates easy cases, attempts hard ones. Fix escalation calibration?
A — explicit escalation criteria + few-shot examples of escalate-vs-resolve.
Meta-rules 2 & 4: proportionate first fix; B (self-confidence) is miscalibrated on hard cases; C over-engineers; D (sentiment) ≠ complexity.

Q4 · CLAUDE CODE Team-wide `/review` command on clone/pull. Where?
A — `.claude/commands/` in the repo (version-controlled, shared).
B is personal. C (CLAUDE.md) is context, not commands. D (`.claude/config.json` commands array) doesn't exist.

Q5 · CLAUDE CODE Monolith → microservices across dozens of files, service-boundary decisions. Approach?
A — plan mode to explore/design before changing.
Architectural + multi-file + multiple approaches = plan mode. D wrongly defers ("switch only if complexity emerges") when complexity is already stated.

Q6 · CLAUDE CODE Test files spread throughout, must share conventions regardless of location. Most maintainable?
A — `.claude/rules/` with glob frontmatter (`**/*.test.tsx`).
B relies on inference. C requires manual invocation (not automatic). D (subdirectory CLAUDE.md) is directory-bound, can't cover scattered files.

Q7 · RESEARCH Reports cover only visual arts; logs show coordinator decomposed into digital art / graphic design / photography. Root cause?
B — coordinator's decomposition too narrow.
Meta-rule 8: the evidence implicates the coordinator; subagents executed their (wrong) assignments correctly. A/C/D wrongly blame working downstream agents.

Q8 · RESEARCH Web-search subagent times out. Best error propagation?
A — structured error context (failure type, attempted query, partial results, alternatives).
B hides context behind a generic status. C suppresses the error as fake success. D terminates the whole workflow.

Q9 · RESEARCH Synthesis needs frequent verification; 85% simple fact-checks, +40% latency. Best fix?
A — give synthesis a scoped `verify_fact` tool for simple lookups; keep complex cases via coordinator.
Meta-rule 5 (least privilege). B batching creates blocking dependencies. C over-provisions all web tools. D speculative caching can't predict needs.

Q10 · CI `claude "..."` hangs on interactive input. Correct approach?
A — add `-p` (`--print`) for non-interactive mode.
B (`CLAUDE_HEADLESS`) and D (`--batch`) don't exist. C (`< /dev/null`) is a workaround, not the documented fix.

Q11 · CI Move both a blocking pre-merge check and an overnight report to Batch API for 50% savings?
A — batch the overnight report only; keep real-time for the blocking pre-merge check.
No SLA on batch → unfit for blocking. B "often faster" is unacceptable for blocking. C's ordering fear is false (`custom_id`). D adds needless complexity.

Q12 · CI 14-file single-pass review is inconsistent/contradictory. Restructure?
A — per-file passes for local issues + a separate cross-file integration pass.
Meta-rule 6: attention dilution. B shifts burden. C (bigger context) doesn't fix attention quality. D (2-of-3 consensus) suppresses real bugs.

Rapid-Review Cheat Sheet

Trigger → Answer reflexes

When the question says…	Reach for…
"must always" / financial / identity / compliance	Hook or prerequisite gate (programmatic), forced `tool_choice`
"first step" / "most effective" to fix misrouting	Rewrite tool descriptions
Inconsistent format / ambiguous judgment	2–4 few-shot examples with reasoning
False positives in review	Explicit categorical criteria (not "be conservative")
Guaranteed structured output	`tool_use` + JSON schema; `tool_choice:"any"`
Model fabricates missing fields	Make fields optional/nullable; add `"unclear"` / `"other"`
Retry won't help	Info absent from source (vs format error = retry works)
Overnight / weekly / non-blocking + cost	Message Batches API (50%, no SLA, `custom_id`)
Blocking / pre-merge / developer waiting	Synchronous API (never batch)
Non-interactive CI run	`-p` / `--print`
Structured CI findings for PR comments	`--output-format json` + `--json-schema`
Convention for a file type across many dirs	`.claude/rules/` + glob frontmatter
Team-shared command / MCP server	`.claude/commands/` · `.mcp.json`
Personal / experimental	`~/.claude/commands/` · `~/.claude.json`
Architectural / multi-file / many approaches	Plan mode
Single-file fix, clear scope	Direct execution
Spawn subagents	`Task` tool; `allowedTools` includes `"Task"`
Parallel subagents	Multiple `Task` calls in one response
Pass findings to a subagent	Put them explicitly in its prompt (no auto-inherit)
Subagent failed	Structured error context; local recovery first
Coverage missing a whole subtopic	Coordinator decomposition too narrow
Reviewing its own generated code	Independent review instance
Contradictory review over many files	Per-file + integration passes (not bigger context)
Numbers/dates lost over a long chat	Persistent "case facts" block outside the summary
Key info dropped from middle of input	Put findings first + section headers
Verbose tool output filling context	Trim to relevant fields before accumulation
Multiple record matches	Ask for another identifier
Customer asks for a human	Escalate immediately
Policy silent on the request	Escalate (policy gap)
97% accuracy, automate?	Segment accuracy by doc type + field first; stratified sampling
Conflicting stats from good sources	Annotate both with attribution; don't pick one
Timestamps look contradictory	Require publication/collection dates
Verbose skill output pollutes chat	SKILL.md `context: fork`
Prevent destructive skill actions	SKILL.md `allowed-tools`
Loop termination	`stop_reason == "end_turn"` only
Normalize heterogeneous tool result formats	`PostToolUse` hook

Distractor patterns that are almost always WRONG

- Train a classifier / deploy a separate ML model / build a routing layer — when a prompt or description fix is untried.
- Use the model's self-reported confidence to route hard cases.
- Use sentiment as a proxy for complexity.
- Bigger model / larger context window to fix attention quality or contradictory reviews.
- Consensus voting (2-of-3) that suppresses intermittently-caught real bugs.
- Prompt instructions ("be conservative", "mandatory") when the requirement is deterministic.
- Silently suppress errors, or kill the whole workflow on one subagent failure.
- Generic error strings ("operation failed", "search unavailable").
- Consolidate tools / rebuild architecture when asked only for a "first step."
- Reference non-existent features (`CLAUDE_HEADLESS`, `--batch`, `.claude/config.json` commands array).
- Defer plan mode "until complexity emerges" when complexity is already stated.

Number & fact anchors to memorize

- Batch API: **50% cost**, **24h** max window, **no SLA**, `custom_id`, **no multi-turn tool calling**.
- Too many tools example: **18 vs 4–5** degrades selection.
- Few-shot count: **2–4** targeted examples.
- Refund gate example threshold: **> $500** → block/escalate via hook.

- Exam: **60 items**, **4 of 6 scenarios**, **120 min**, pass **720/1000**.
- Scopes: project = shared/VCS (`.claude/` , `.mcp.json`); user = personal (`~/.claude/` , `~/.claude.json`).
- `stop_reason` : `"tool_use"` = continue; `"end_turn"` = stop.
- `tool_choice` : `"auto"` (may text) / `"any"` (must call some tool) / forced (must call named tool).

> **Exam-day approach.** Read the whole scenario stem before the options — the log detail (like "decomposed into three visual-arts subtasks") usually names the culprit. Identify the requirement type (deterministic vs best-effort), then eliminate: cross out non-existent features, over-engineered answers, and anything relying on sentiment or self-confidence. When two answers remain, pick the one that is (a) proportionate to a "first step," (b) fixes the root cause, and (c) preserves structure/metadata. You were 31 points short — these eliminations are worth far more than that.